REPORTE DE PRÁCTICA

Creación y ejecución de una aplicación básica con Jetpack Compose(Video 1)

Proyecto	HelloWorldCompose
IDE	Android Studio
Lenguaje	Kotlin
Framework de interfaz	Jetpack Compose
Sistema de pruebas	Android Emulator
Dispositivo virtual	Pixel 4 — Android 15 / API 35
Sistema operativo	Linux

Nombre del alumno: Daniel Jimenez Pluma

Materia / grupo: Desarrollo de aplicaciones moviles/04_01

Docente: Miguel Angel Montoya Cerro

Fecha: 25/Septiembre/2026



1. Introducción

La práctica consistió en preparar un proyecto básico de Android utilizando Android Studio, Kotlin y Jetpack Compose, y posteriormente comprobar su funcionamiento mediante un dispositivo virtual Android. El objetivo práctico fue conseguir que una aplicación sencilla pudiera compilarse, instalarse y ejecutarse dentro del emulador.

Durante el proceso surgieron problemas relacionados con la ejecución del emulador. Por ello, además del desarrollo de la interfaz, fue necesario revisar la configuración del dispositivo virtual, comprobar la aceleración de virtualización de Linux y finalmente iniciar el emulador desde la Terminal.

2. Objetivos

Objetivo general: desarrollar y ejecutar una aplicación básica de Android mediante Jetpack Compose.

Objetivos específicos:

- Configurar un proyecto de Android Studio llamado HelloWorldCompose.
- Trabajar con una interfaz declarativa mediante funciones @Composable.
- Crear una pantalla sencilla utilizando Column y Text.
- Configurar un dispositivo virtual Android para realizar pruebas.
- Diagnosticar problemas de ejecución del Android Emulator.
- Comprobar la aceleración KVM del sistema Linux.
- Ejecutar correctamente la aplicación dentro del emulador.

3. Herramientas y tecnologías utilizadas

- Android Studio: entorno de desarrollo utilizado para crear, editar, compilar y ejecutar el proyecto.
- Kotlin: lenguaje utilizado para escribir el código de la aplicación.
- Jetpack Compose: toolkit de Android utilizado para construir la interfaz de usuario mediante código.
- Android Emulator: herramienta que permite ejecutar un dispositivo Android virtual.
- KVM: mecanismo de virtualización utilizado en Linux para mejorar el rendimiento del emulador.
- Pixel 4 virtual: dispositivo utilizado finalmente para ejecutar la aplicación.

4. Configuración inicial del proyecto

El proyecto se denominó HelloWorldCompose. La actividad principal utiliza ComponentActivity y establece el contenido mediante setContent. La pantalla se construye con una función composable llamada PrimeraPantalla().

Código principal utilizado

```
package com.example.helloworldcompose

import android.os.Bundle
```

```
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            PrimeraPantalla()
        }
    }
}

@Composable
fun PrimeraPantalla() {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = "¡Hola Mundo!",
            fontSize = 32.sp
        )

        Text(
            text = "Mi primera aplicación con Jetpack Compose",
            fontSize = 18.sp,
            modifier = Modifier.padding(top = 16.dp)
        )
    }
}

@Preview(showBackground = true)
@Composable
fun PrimeraPantallaPreview() {
    PrimeraPantalla()
}
```

5. Descripción de la interfaz

La función PrimeraPantalla() utiliza un Column para colocar los elementos verticalmente. El modificador fillMaxSize() ocupa el espacio disponible y padding(24.dp) agrega margen interno. horizontalAlignment centra los elementos horizontalmente y verticalArrangement los centra verticalmente.

- El primer Text muestra “¡Hola Mundo!” con un tamaño de 32.sp.
- El segundo Text muestra “Mi primera aplicación con Jetpack Compose” con un tamaño de 18.sp.
- Se utiliza padding superior de 16.dp entre ambos textos.
- La anotación @Preview permite visualizar la interfaz desde Android Studio sin ejecutar el dispositivo.

6. Problemas encontrados durante la ejecución

Después de preparar el proyecto, la compilación llegó a completarse correctamente. El problema principal apareció al intentar ejecutar la aplicación porque no había un dispositivo disponible. Posteriormente se creó un Pixel 8 virtual, pero el proceso del emulador terminaba antes de iniciar correctamente.

Mensaje observado en Android Studio:

```
Error running 'app'
The emulator process for AVD Pixel_8 has terminated.
```

Se probaron diferentes configuraciones del AVD, entre ellas Wipe Data, Cold Boot, aceleración gráfica mediante Software y ajustes de memoria. Como el proyecto ya compilaba correctamente, se mantuvo el código de la aplicación sin modificaciones innecesarias.

7. Diagnóstico del entorno mediante Terminal

Para separar el problema del proyecto del problema del emulador, se utilizó la Terminal de Linux. Primero se comprobó la aceleración de virtualización con el comando:

```
~/Android/Sdk/emulator/emulator -accel-check
```

El resultado fue:

```
accel:
0
KVM (version 12) is installed and usable.
accel
```

Este resultado confirmó que KVM estaba instalado y disponible para el emulador. Por lo tanto, no era necesario solucionar un problema de virtualización de KVM.

También se ejecutó el Pixel 8 desde Terminal y se observaron advertencias relacionadas con la memoria del sistema. El emulador indicó una memoria de sistema sugerida de 16384 MiB, mientras que en ese momento había aproximadamente 7768 MiB disponibles. Esto llevó a buscar una configuración más ligera para la práctica.

8. Creación y prueba del Pixel 4

Como alternativa se creó un segundo dispositivo virtual: Pixel 4 utilizando una imagen de Android 15 / API 35. Esta configuración resultó más adecuada para el equipo utilizado.

Al ejecutar el Pixel 4 desde Terminal se obtuvo la comprobación de compatibilidad del sistema:

```
Ok: Disk space requirements to run avd: `Pixel_4` are met
Ok: Hardware GPU compatibility checks are not required
Ok: System requirements to run avd: `Pixel_4` are met
```

Finalmente, el emulador se inició mediante:

```
~/Android/Sdk/emulator/emulator -avd Pixel_4 -verbose
```

La opción -verbose permitió observar información detallada durante el arranque. En esta ocasión el Pixel 4 inició correctamente y la aplicación HelloWorldCompose apareció en la pantalla del dispositivo.

9. Evidencia del resultado



10. Procedimiento para iniciar nuevamente el emulador

Una vez creado el AVD Pixel 4, se puede iniciar directamente desde la Terminal cuando sea necesario. El procedimiento recomendado es:

- Abrir la Terminal de Linux.
- Ejecutar el comando ~/Android/Sdk/emulator/emulator -avd Pixel_4.
- Esperar a que aparezca la pantalla de inicio de Android.
- Mantener la Terminal abierta mientras el emulador esté ejecutándose.
- Abrir Android Studio y ejecutar la aplicación sobre el Pixel 4 que ya está encendido.
- No iniciar simultáneamente el mismo AVD desde Terminal y desde Device Manager.

Si el emulador queda bloqueado o existe un proceso anterior, se puede cerrar el proceso del Emulator desde la Terminal con:

```
pkill -f emulator
```

Después de cerrar el proceso, se recomienda iniciar el Pixel 4 una sola vez. Durante la práctica se observó que intentar abrir el mismo AVD mientras ya estaba ejecutándose producía el mensaje de función experimental para múltiples instancias del mismo AVD.

11. Resultado obtenido

El resultado final fue satisfactorio: el proyecto HelloWorldCompose compiló correctamente y la aplicación se ejecutó dentro de un dispositivo Android virtual. La interfaz mostró el mensaje “¡Hola Mundo!” y el texto “Mi primera aplicación con Jetpack Compose”, confirmando que la actividad principal y la interfaz Compose estaban funcionando.

- El emulador Pixel 4 inició correctamente.
- La aplicación HelloWorldCompose se instaló/ejecutó en el dispositivo virtual.
- La pantalla de la aplicación mostró los dos textos definidos en PrimeraPantalla().
- El entorno de virtualización KVM fue comprobado como funcional.
- El problema inicial quedó aislado del código de la aplicación.

12. Conclusión

La práctica permitió conocer el flujo básico de desarrollo y prueba de una aplicación Android con Jetpack Compose. Además de construir una interfaz sencilla, se aprendió a identificar la diferencia entre un problema de compilación del proyecto y un problema del dispositivo de prueba.

La dificultad principal estuvo relacionada con el arranque del Android Emulator. El diagnóstico mediante Terminal fue especialmente útil porque permitió comprobar KVM, revisar los requisitos del dispositivo y observar directamente los mensajes generados por el emulador. Finalmente, el uso de un Pixel 4 con Android 15 / API 35 permitió ejecutar la aplicación correctamente.

Como resultado, se obtuvo una aplicación funcional de tipo “Hola Mundo” desarrollada con Jetpack Compose y se dejó establecido un procedimiento para volver a iniciar el emulador mediante Terminal cuando sea necesario.

13. Comandos utilizados durante la práctica

Comprobar virtualización: ~/Android/Sdk/emulator/emulator -accel-check

Listar AVD disponibles: ~/Android/Sdk/emulator/emulator -list-avds

Iniciar Pixel 4: ~/Android/Sdk/emulator/emulator -avd Pixel_4

Iniciar Pixel 4 con información detallada: ~/Android/Sdk/emulator/emulator -avd Pixel_4 -verbose

Cerrar procesos del Emulator: pkill -f emulator